



北京理工大學珠海學院

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

基于 ResNet50 网络实现 Cifar100 图像分类

学 院： 数理与土木工程学院

课 程： 大数据工具箱

班 级： 数据科学与大数据技术 2 班

姓 名： 肖宇涵 学 号： 211205101786

中国·珠海

2023 年 12 月 27 日

基于 ResNet50 网络实现 Cifar100 图像分类

摘 要

本研究旨在探究 ResNet50 网络在 CIFAR-100 数据集上进行图像分类的性能。为此，我们采用了预训练的 ResNet50 模型，并对其最后一层进行了修改，以适应 CIFAR-100 数据集的 100 个类别。数据预处理包括调整图像大小至 224x224 像素、随机水平翻转和标准化。实验中使用了基于梯度下降的优化器，并设置多个学习率直到找到模型的最优解。

分别设立多个训练周期，模型的性能通过 TensorBoard 进行监测和记录，包括训练损失、训练准确率、测试准确率以及错误率。实验结果显示，该模型在 CIFAR-100 数据集上表现出良好的分类能力。具体而言，随着训练过程的进行，训练损失显著降低，而训练和测试的准确率均有所提高。此外，还绘制了训练损失、准确率以及错误率随时间的变化图，以直观展示模型性能的进步。

充分证明了在合适的参数配置下，ResNet50 网络能够有效地应用于复杂的图像分类任务中。最终，我们将训练好的模型保存，以便于后续的使用和分析。

关键词：ResNet50 神经网络 CIFAR-100 图像分类

目 录

1	引言	1
2	相关研究	1
3	基本理论	2
3.1	卷积神经网络相关原理	2
3.2	典型的 CNN 的构成	3
3.2.1	卷积层——提取特征	4
3.2.2	池化层（下采样）——数据降维，避免过拟合	5
3.2.3	全连接层——输出结果	5
3.3	ResNet 卷积神经网络基本结构	5
3.3.1	网络结构	6
3.3.2	residual 模块	7
3.3.3	batch normalization	8
4	图像数据集的介绍及预处理	9
4.1	数据预处理	10
5	ResNet50 实现 Cifar100 图像分类	11
5.1	ResNet50 模型架构	11
5.2	模型训练过程	12
5.2.1	模型调整	12
5.2.2	训练设置	13
5.2.3	性能监测	13
6	模型训练结果	13
6.1	CIFAR-100 数据集上的分类结果	13
6.1.1	第一次训练	13
6.1.2	第二次训练	13
7	模型的网页部署	16
8	分析与讨论	17
8.1	实验结果分析	17
8.2	模型的优势与局限性	17
8.3	与相关研究的对比	17

9 研究总结	18
9.1 研究贡献	18
9.2 未来研究方向或改进点	18
参考文献	19
附录	21

1 引言

图像分类作为计算机视觉领域的一个核心问题，长期以来一直是学术研究和工业应用的热点。随着深度学习技术的快速发展，特别是卷积神经网络（CNN）的广泛应用，图像分类的性能得到了显著提升。在众多的 CNN 架构中，ResNet（残差神经网络）因其独特的“跳跃连接”设计而脱颖而出，有效解决了深层网络中的梯度消失问题，从而使得网络能够达到更深的层次，以提高分类精度。

CIFAR-100 数据集是一个公认的图像分类基准测试集，包含 100 个类别，每个类别 600 张图像，其中 500 张用于训练，100 张用于测试。由于其多样性和复杂性，CIFAR-100 成为评估图像分类模型性能的理想选择。

本研究旨在探究预训练的 ResNet50 模型在 CIFAR-100 数据集上进行图像分类的效果。考虑到数据集的复杂性和模型的深度，我们对 ResNet50 模型进行了适当的调整，并采用了先进的数据预处理技术以提高模型的泛化能力。此外，通过细致的超参数调整和优化策略，我们旨在实现对 CIFAR-100 数据集的高效准确分类。

随着人工智能技术在各个领域的应用日益增加，对高效和准确的图像分类算法的需求也在不断增长。我们希望本研究能为深度学习在图像分类方面的应用提供有价值的见解和参考。

2 相关研究

在深度学习和计算机视觉的发展进程中，图像分类一直是一个关键的研究领域。早期的图像分类方法依赖于手工提取的特征和传统的机器学习算法，如支持向量机（SVM）。然而，这些方法在面对复杂和高维度的图像数据时往往表现不佳。随着深度学习的兴起，尤其是卷积神经网络（CNN）的引入，图像分类的性能得到了显著提升。

AlexNet 的成功标志着深度 CNN 在图像分类任务中的潜力。随后，研究者们提出了多种改进的 CNN 架构，如 VGGNet、GoogLeNet 和 ResNet。特别是 ResNet，其引入的残差学习框架为训练更深层次的网络奠定了基础，并在多个基准测试中取得了突破性的结果。ResNet 的这一成就主要归功于它能解决深层网络中的梯度消失问题，从而使得网络能够学习更复杂的特征。

在对特定数据集进行分类的研究中，CIFAR-100 作为一个具有挑战性的数据集，吸引了大量的关注。这个数据集由 100 个类别的小图像组成，每个类别有 600 个图像。由于其多样性和类间相似性，CIFAR-100 成为了评估图像分类模型的一个重要标准。

近年来，很多研究集中在如何改进模型的结构和训练策略以适应 CIFAR-100 这样的复杂数据集。例如，一些研究引入了数据增强技术，如图像裁剪和翻转，以提高模型的泛化能力。其他研究则探讨了不同的优化算法，例如使用动量和自适应学习率的随机梯度下降（SGD），以加速模型的训练过程和提高其最终性能。

综上所述，虽然深度学习在图像分类方面已取得显著进展，但针对特定复杂数据集的研究，尤其是如何有效地调整和优化现有模型以提高分类性能，仍然是一个活跃的研究

究领域。本研究基于这一背景，通过在 CIFAR-100 数据集上应用并调整 ResNet50 模型，旨在对这一领域作出贡献。

3 基本理论

3.1 卷积神经网络相关原理

神经网络其实是一种模拟人脑的神经系统。由很多带有具体信息的节点组成像网一样的模型，这些节点称为网络中的神经元、处理单元。本质上，通过特定的方法把众多包含信息的神经元及运算关系组合连接起来组成的网络就形成了我们所说的神经网络。

“其中， x 代表输入有信息的物理量， x_n 表示有信息的第 n 个分物理量， w 表示神经元的权重系数”。分析神经网络的时候，我们只在网络里随机拿一个节点出来，摄取神经网络中的一个特征物理量数据。在初始化时， w 权重系数会在训练参数的过程中一步步的修正里面的参数。 b 是偏差参数，若 b 高，则处理结果会很快的传送出来，若 b 低，则结果很难传输。 f 为激活参数，一般为非线性函数，在神经网络的分析中，加入一定量的非线性函数因素，能使网络很快的处理比较复杂的问题。 Y 是输出结果。这些相互连接的神经元在做矩阵运算时，每一个神经元的输出信息代“乘以权重值 w ”，然后加上一个偏差值 b ，最后经过一个激活函数的处理就变成了下一层神经元的输入结果。我们可以用公式表示：

$$y = f \left(\sum_{i=1}^n X_i w_i + b \right) \quad (1)$$

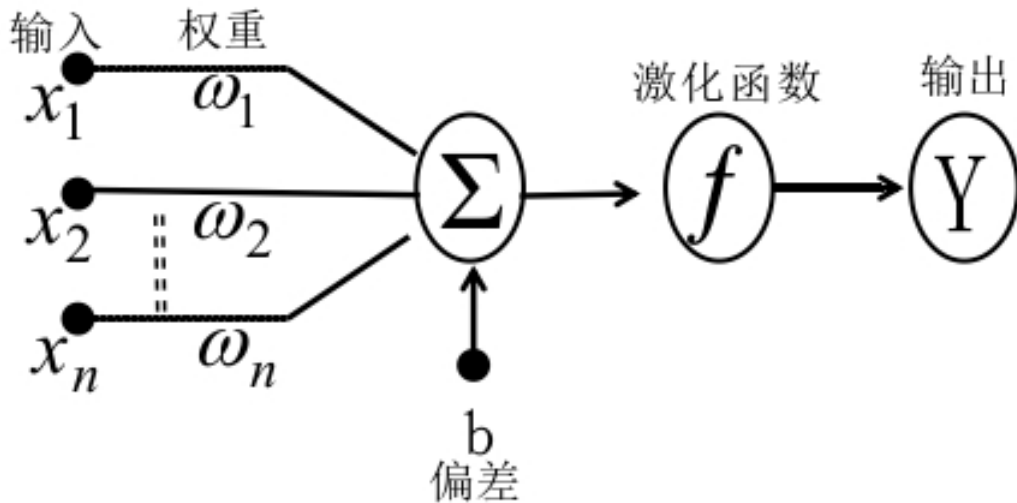


图 1: 神经元计算示意图

众多的神经元通过特定的函数就能构成一个完整的神经网络。在函数的分析处理中，形式多样，结构复杂。就比如我们在思考一个问题的时候，会把发生的地点，任务

人物，时间，思考的方向等等因素统统考虑进去。这些不同的因素就像神经元一样，会把所有的因素都在隐藏在大脑中的神经元去计算，最后得到一个处理的整合数据。网络结构图如下所示：

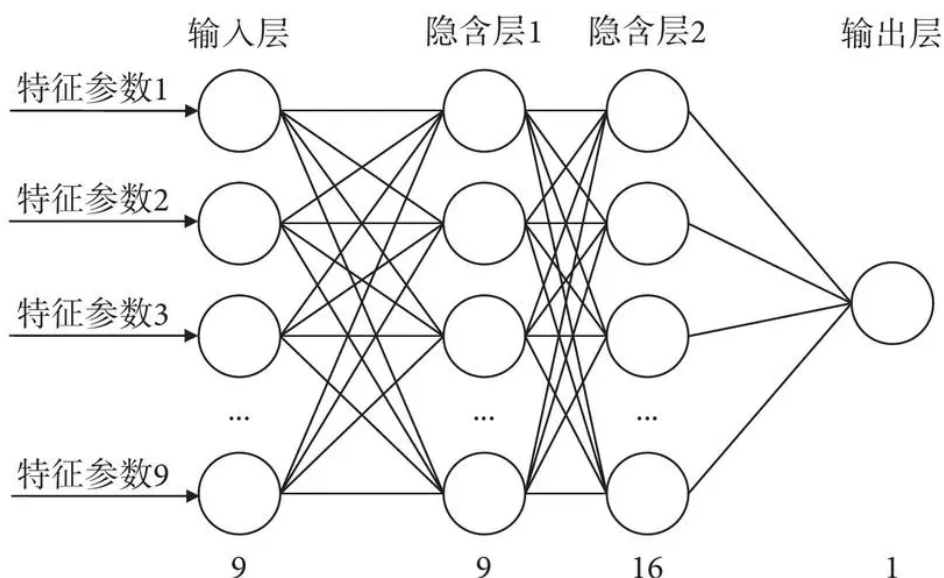


图 2: Caption

在 CNN 出现之前，图像对于人工智能来说是一个难题，有两个原因：

- 图像需要处理的数据量太大，导致成本很高，效率很低。
- 图像在数字化的过程中很难保留原有的特征，导致图像处理的准确率不高。

卷积神经网络-CNN 解决的第一个问题就是“将复杂问题简化”，把大量参数降维成少量参数，再做处理。

CNN 用类似视觉的方式保留了图像的特征，当图像做翻转，旋转或者变换位置时，它也能有效的识别出来是类似的图像。

3.2 典型的 CNN 的构成

典型的 CNN 由以下三个部分构成：

- 卷积层
- 池化层
- 全连接层

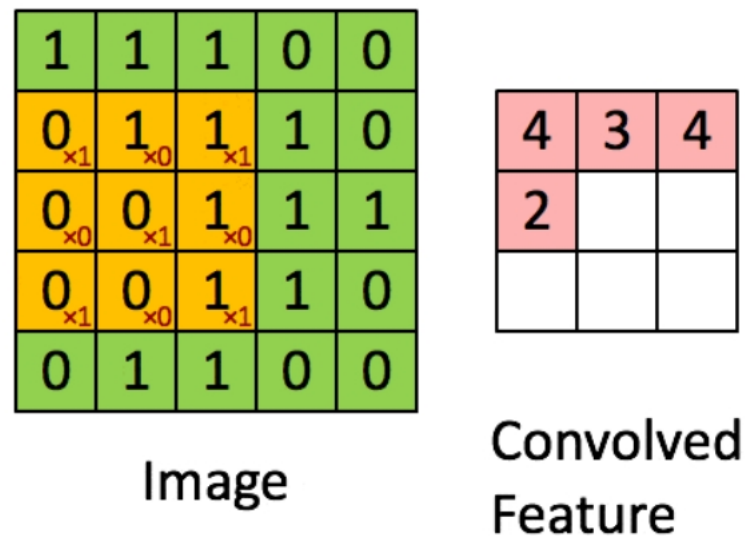


图 3: 卷积提取特征

3.2.1 卷积层——提取特征

卷积层的运算过程可以用一个卷积核扫描整张图片来理解。这个过程可视为使用一个过滤器（卷积核）来过滤图像的各个小区域，从而得到这些区域的特征值。

具体应用中，通常有多个卷积核，每个卷积核代表一种图像模式。以下是 25 种不同卷积核的示例。

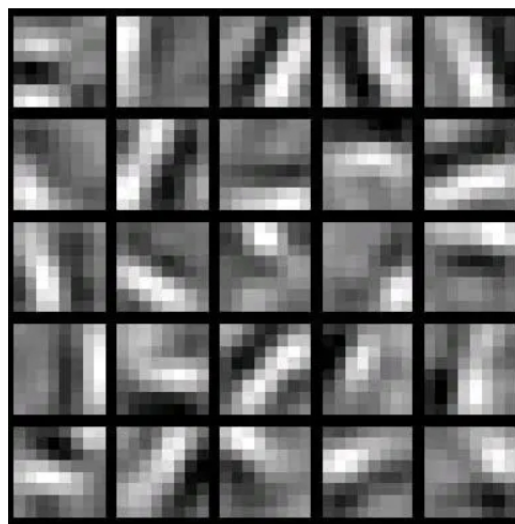


图 4: 25 种不同的卷积核的示例

3.2.2 池化层（下采样）——数据降维，避免过拟合

池化层可以将一幅大图像缩小，同时保留重要信息。它通过减小网络规模和参数，起到降维作用。

Lp 池化（Lp pooling）定义如下：

$$A_k^l(i, j) = \left[\sum_{x=1}^f \sum_{y=1}^f (A_k^l(s_0 i + x, s_0 j + y))^p \right]^{\frac{1}{p}}$$

步长 s_0 、像素 (i, j) 的含义与卷积层相同， p 是预设参数。当 $p = 1$ 时，进行均值池化（average pooling）；当 $p = \infty$ 时，进行最大池化（max pooling）。以下是最大池化的过程。

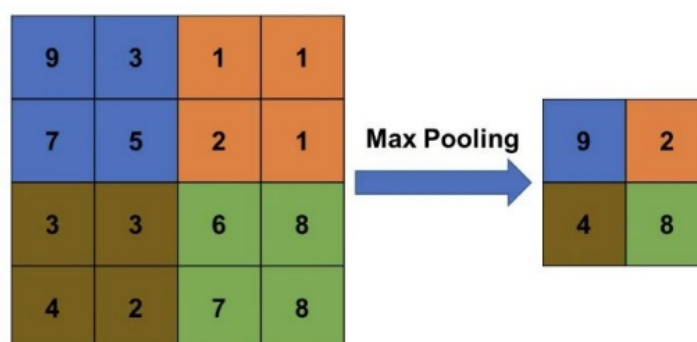


图 5: 最大池化

3.2.3 全连接层——输出结果

经过卷积层和池化层处理后的数据输入到全连接层，以得到最终结果。全连接层需要降维后的数据，否则数据量过大，导致计算成本高，效率低。

典型的 CNN 不仅仅是以上提到的三层结构，而是多层结构，例如接下来将介绍的 ResNet 卷积神经网络。

3.3 ResNet 卷积神经网络基本结构

ResNet 是一种以卷积结构为主，搭建起来的深度网络。将图片作为网络的输入，ResNet 网络将自动提取特征，并且对图像的变形等具有高度不变性。其具有卷积神经网络的卷积层，池化层，激活层，全连接层等一般结构，并且引入了跳连结构作为核心单元。下图是 ResNet 卷积神经网络的大致结构。

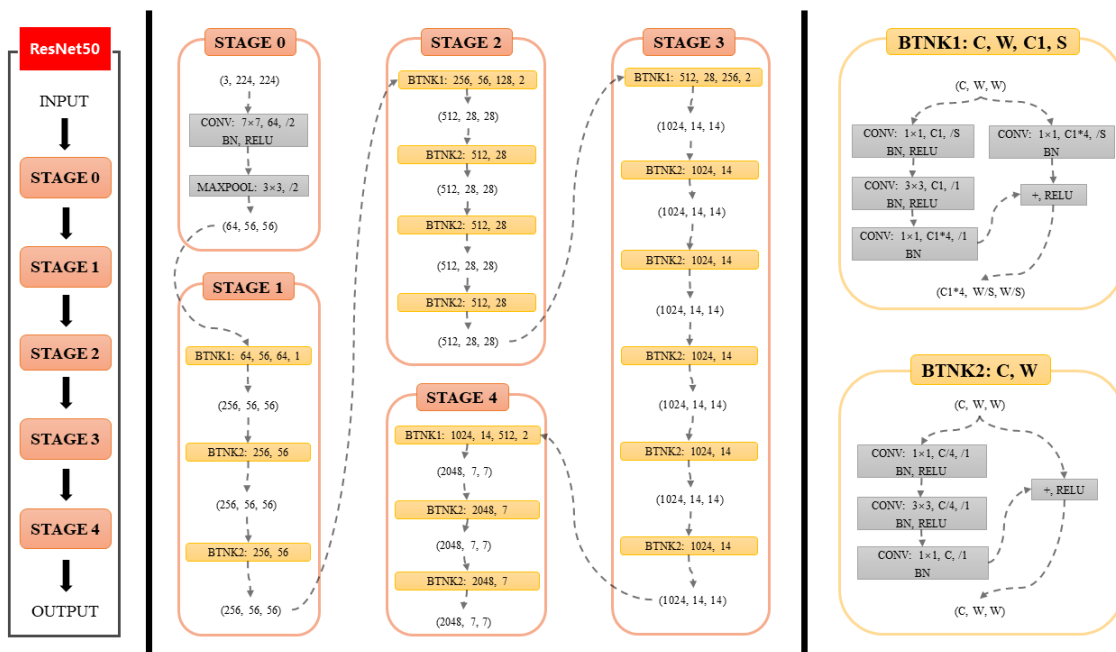


图 6: ResNet 卷积神经网络基本结构图

ResNet 卷积神经网络 2015 年由微软实验室的何恺明、张祥雨、任少卿、孙剑提出的，网络的亮点主要有：

- 超深的网络结构（可以突破 1000 层）；
- 提出 residual 模块；
- 使用 batch normalization 加速训练（丢弃 dropout）；

3.3.1 网络结构

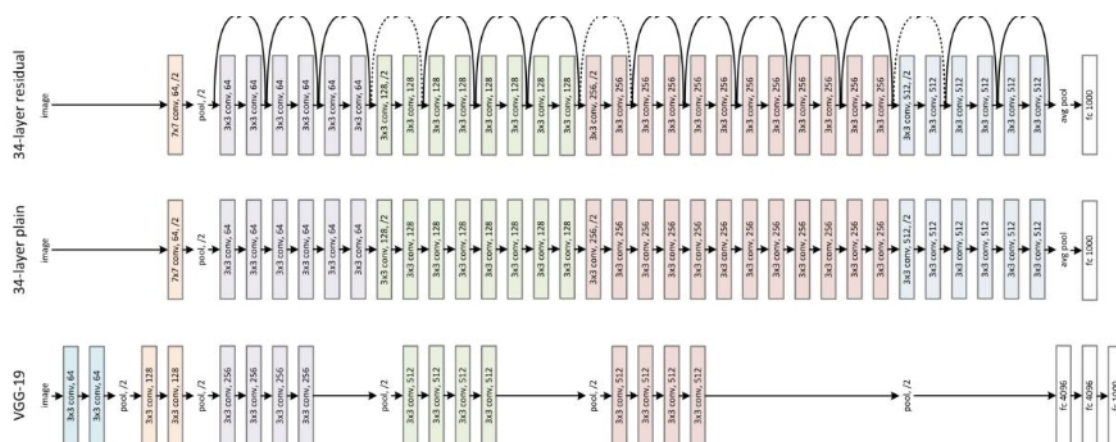


图 7: 网络结构

可以看到 ResNet 其实就是对 residual 模块的反复堆叠，随着网络层数的增加，深层网络中主要会出现如下两个问题而导致网络效果变差：

1. 梯度消失或者梯度爆炸；
2. 退化问题；

其中梯度消失或者梯度爆炸主要通过数据预处理、权重初始化以及 **batch normalization** 方法解决，而退化问题主要是通过本文提出的 **residual** 模块解决。

3.3.2 residual 模块

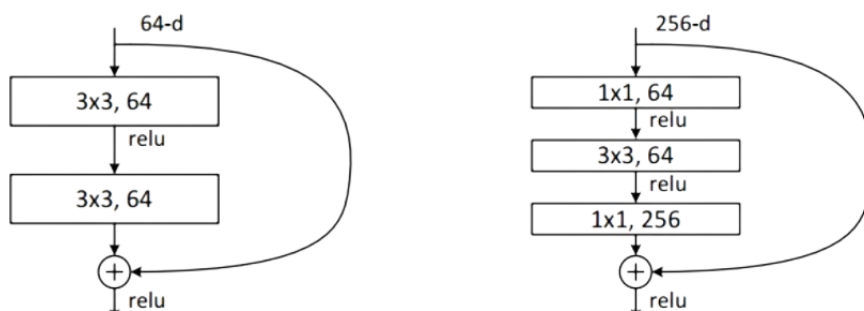


图 8: residual 模块

residual 模块主要解决了深层网络出现的退化问题，退化问题具体表现就是层数深的网络反而没有层数浅的好。左侧为 34 层网络的 **residual** 模块，右侧为 50、101、152 层网络的 **residual** 模块。右侧 **residual** 模块的主要特点是增加 1×1 卷积核来进行升维和降维，减少网络参数量。

add 要求整个特征矩阵的长宽和深度都相同，而 **concat** 仅仅要求长宽相同，而深度可以不同，因此 **concat** 就是按照深度方向进行拼接的 **add** 和 **concat** 操作是可以相互转换的，如下图所示：

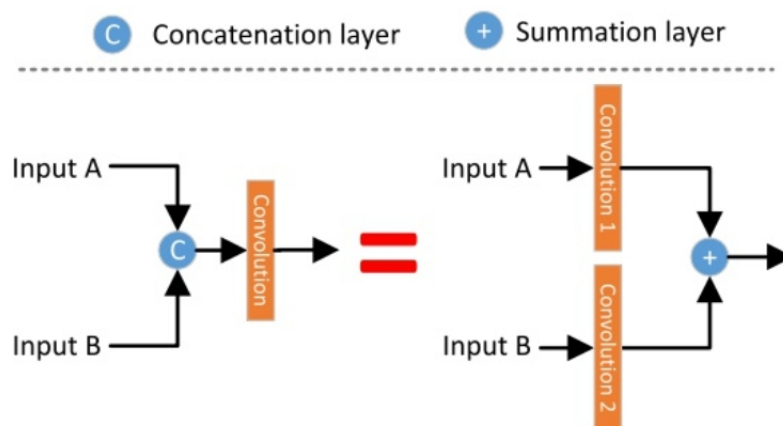


图 9: add 和 concat 相互转换

由此可见，**add** 其实可以相当于 **concat** 之后对应通道共享同一个卷积核，**add** 相当于加了一种先验，当两路输入可以具有“对应通道的特征图语义类似”（可能不太严谨）的性质的时候，可以用 **add** 来替代 **concat**，这样更节省参数和计算量（**concat** 是 **add** 的 2 倍）。

3.3.3 batch normalization

Batch Normalization 是 2015 年的论文《Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift》提出的，其目的是使得我们同一通道的 feature map 满足均值为 0，方差为 1 的分布规律，batch normalization 的具体操作如下：假设小批量输入为 $B = \{x_{1...m}\}$ ，学习的参数为 γ 和 β ，那么操作主要有如下四个步骤：

1. 求得小批量数据的均值：

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

2. 求得小批量数据的方差：

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

3. 进行归一化操作：

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

4. 进行尺度与位移变换：

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i)$$

其中， ϵ 是为了数值稳定性而添加的小常数。

其中 batch normalization 的结果 $y_i = \text{BN}_{\gamma, \beta}(x_i)$ 的具体操作可以如下图所示：

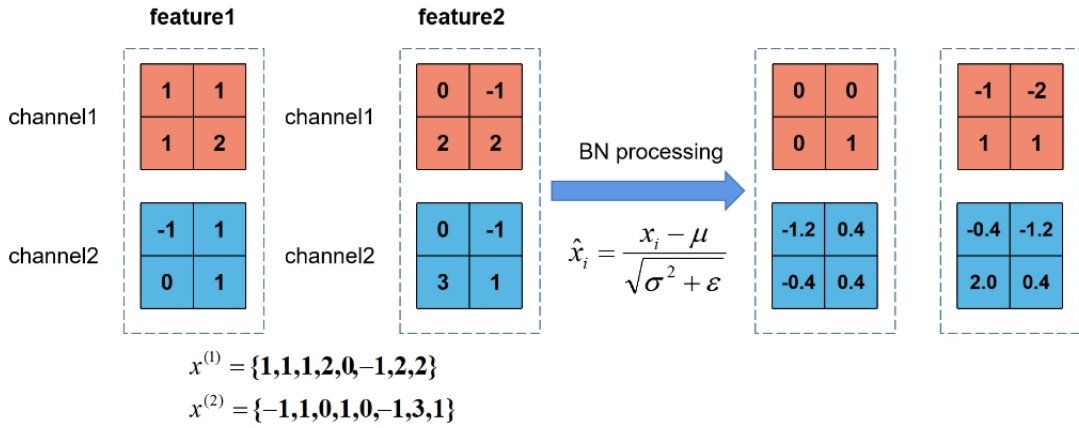


图 10: batch normalization 的结果具体操作

这里值得注意的是：

γ 和 β 两个参数是在训练过程中学习得到的，作用是抵消由于 Batch Normalization 将特征归一化到 0-1 后导致函数曲线变得线性的问题，相当于是回复出原始所学到的特征；

在训练过程中，batch size 要尽可能设置得大一些，这样 Batch Normalization 层才更容易学到数据的分布规律；

Batch Normalization 层通常放在卷积层和激活层之间，且卷积层不需要设置 bias，因此有 Batch Normalization 层的话卷积层有无 bias 的结果是一致的。

在训练过程中，均值和方差分别是由该批次内数据相应维度的均值与方差获得的；在推理过程中，均值和方差是基于所有批次的数据计算所得，也就是说数据的均值和方差在训练过程中会被记录下来并不断更新。

4 图像数据集的介绍及预处理

本研究采用的是 CIFAR-100 数据集，这是一个由加拿大高级研究院（CIFAR）发布的公开数据集，广泛用于计算机视觉领域的图像识别任务。它包含 60,000 张 32x32 像素的彩色图像，共分为 100 个类别，每类有 600 张图像。数据集的类别及其子类如见表 1，其中，50,000 张图像被划分为训练集，用于模型训练，而剩余的 10,000 张图像则构成了测试集，用于评估模型性能。这个数据集不仅类别丰富，而且每个类别之间的图像差异性大，这对图像分类算法来说是一个挑战。

表 1: Cifar-100 数据集类别及其子类

Superclass	Classes
Aquatic Mammal	Beaver, Dolphin, Otter, Seal, Whale
Fish	Aquarium Fish, Flatfish, Ray, Shark, Trout
Flowers	Orchids, Poppies, Roses, Sunflowers, Tulips
Food Containers	Bottles, Bowls, Cans, Cups, Plates
Fruit and Vegetables	Apples, Mushrooms, Oranges, Pears, Sweet Peppers
Household Electrical Devices	Clock, Computer Keyboard, Lamp, Telephone, Television
Household Furniture	Bed, Chair, Couch, Table, Wardrobe
Insects	Bee, Beetle, Butterfly, Caterpillar, Cockroach
Large Carnivores	Bear, Leopard, Lion, Tiger, Wolf
Large Man-Made Outdoor Things	Bridge, Castle, House, Road, Skyscraper
Large Natural Outdoor Scenes	Cloud, Forest, Mountain, Plain, Sea
Large Omnivores and Herbivores	Camel, Cattle, Chimpanzee, Elephant, Kangaroo
Medium-Sized Mammals	Fox, Porcupine, Possum, Raccoon, Skunk
Non-Insect Invertebrates	Crab, Lobster, Snail, Spider, Worm
People	Baby, Boy, Girl, Man, Woman
Reptiles	Crocodile, Dinosaur, Lizard, Snake, Turtle
Small Mammals	Hamster, Mouse, Rabbit, Shrew, Squirrel
Trees	Maple, Oak, Palm, Pine, Willow
Vehicles 1	Bicycle, Bus, Motorcycle, Pickup Truck, Train
Vehicles 2	Lawn-Mower, Rocket, Streetcar, Tank, Tractor

每个图像含有一个“fine”标签（表示所属分类）和一个“coarse”标签（所属大类），大小为 32×32 像素。图 13展示了 CIFAR-100 数据集中部分的分类样例图像。

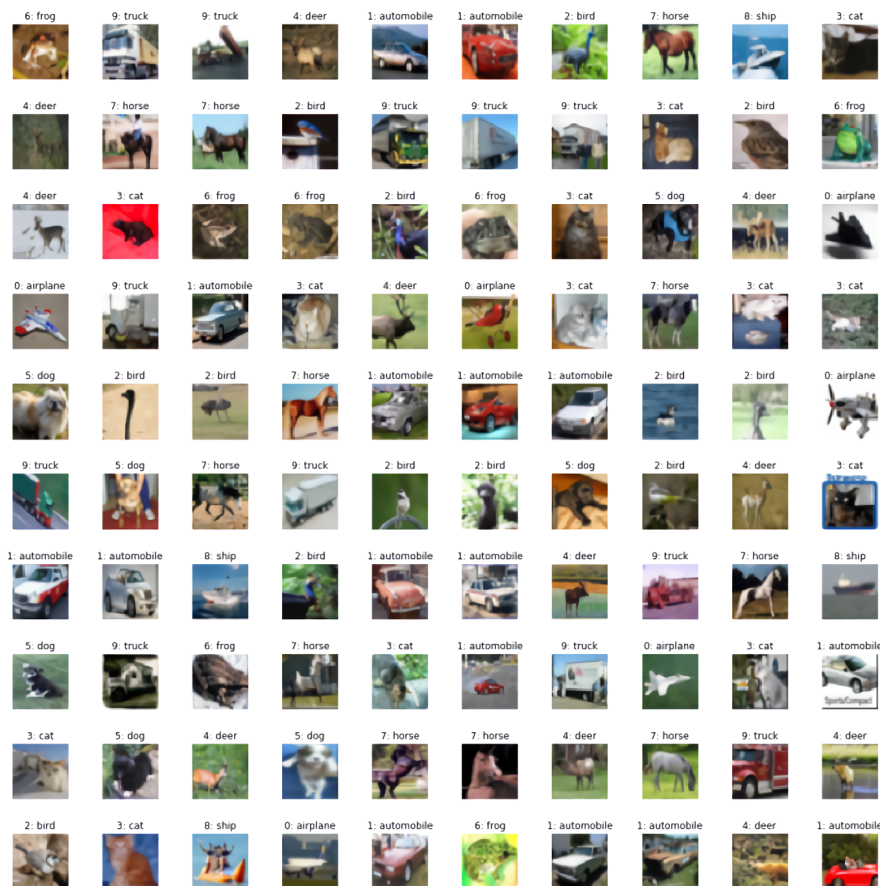


图 11: Cifar-100 数据集中的分类样例图像

4.1 数据预处理

为了提高模型在 CIFAR-100 数据集上的泛化能力，并模拟现实世界条件下的图像变化，我们对训练数据进行了以下预处理和增强操作：

- 大小调整：所有图像首先被调整至 256×256 像素。此操作为后续步骤的随机裁剪留出空间，确保从多样化的图像部分中获取信息。
- 随机裁剪：通过随机裁剪图像至 224×224 像素，我们在空间维度上引入了随机性，有助于模型学习对图像中对象位置变化的鲁棒性。此步骤模拟了对象在视野中可能出现的不同位置。
- 随机水平翻转：图像以 0.5 的概率进行随机水平翻转，增加了模型对镜像变化的不变性，模拟了现实世界中摄像机拍摄方向的多样性。
- 随机旋转：图像被随机旋转 ± 15 度，模仿了真实世界中由于拍摄角度偏差而产生的图像变形。

- 颜色抖动：使用颜色抖动技术，通过 $\pm 20\%$ 的亮度、对比度和饱和度变化，以及 $\pm 10\%$ 的色相偏移来模拟不同的光照条件和颜色变化。
- 标准化：将图像转换为张量后，我们使用 ImageNet 数据集的均值和标准差进行了标准化。此步骤有助于加速模型训练过程，同时减少模型对特定数据集分布的依赖。

上述预处理步骤不仅提高了模型对输入数据变化的鲁棒性，而且也帮助避免了过拟合，这对于在具有高内在复杂度的 CIFAR-100 数据集上训练深度学习模型来说至关重要。

5 ResNet50 实现 Cifar100 图像分类

5.1 ResNet50 模型架构

ResNet50 是一种深度残差网络，它由 50 层卷积层组成。这种网络的核心思想是引入“残差学习”来解决深层网络中的梯度消失问题。在传统的卷积神经网络中，随着层数的增加，网络越来越难以训练。残差网络通过引入跳跃连接，跳跃神经网络中的某些层，并将一层的输出作为下一层的输入，允许信息的直接流动。引入跳跃连接是为了解决不同架构中的不同问题。在 ResNets 的情况下，跳跃连接解决了我们之前解决的退化问题，而在 DenseNets 的情况下，它确保了特征的可重用性，使得深层网络的训练变得更加有效。

如图，具有跳跃连接的神经网络的损失表面更平滑，因此比没有任何跳跃连接的网络收敛速度更快。

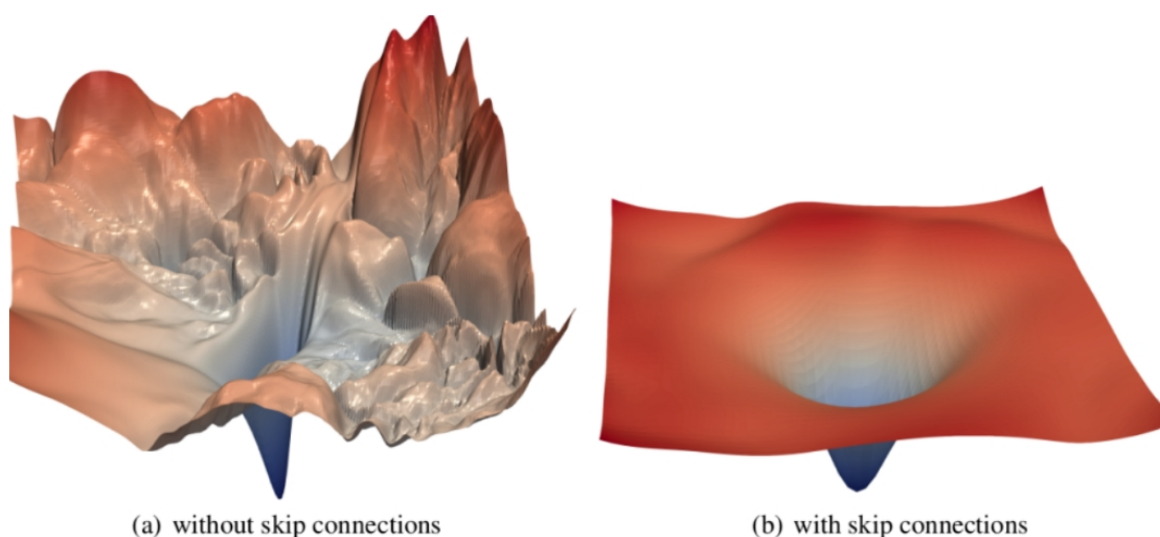


图 12: 跳跃连接

在 ResNet50 中，每个残差块包含几个卷积层，这些卷积层的输出通过跳跃连接添加到块的输入上。这种设计使得网络可以学习到输入和输出之间的残差映射，而不是直

接学习一个映射。这样的架构有助于网络捕获深层次的特征，同时保持训练的稳定性。

如图是利用 TensorFlow 架构中的 TensorBoard 可视化库，将设计出的本系统的 ResNet 的主体网络结构用计算图的形式表示。

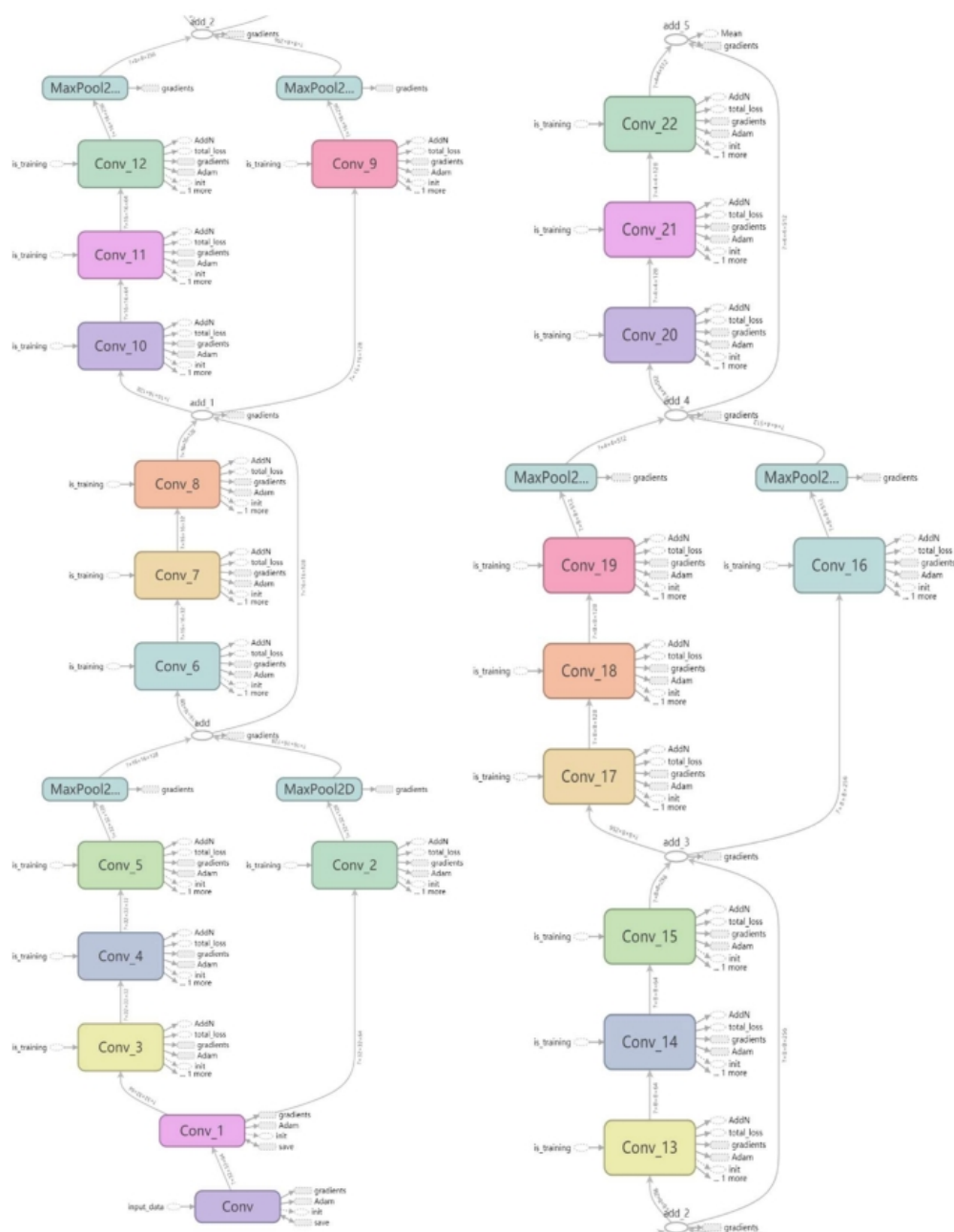


图 13: ResNet 模型架构

5.2 模型训练过程

5.2.1 模型调整

我们使用了预训练的 ResNet50 模型作为起点。由于 CIFAR-100 有 100 个类别，我们替换了原始 ResNet50 模型的最后一个全连接层，将其输出类别数从 1000（ImageNet 类别

数) 调整为 100。

5.2.2 训练设置

我们将模型部署在 GPU 上进行训练，以加速计算过程。使用随机梯度下降 (SGD) 作为优化器，初始学习率设置为 0.001，动量为 0.9。选用交叉熵损失函数，适用于多类分类问题。模型在不同个训练周期内进行训练，每个周期结束时，我们记录了训练损失和准确率。

5.2.3 性能监测

我们使用 TensorBoard 来监控训练过程，包括损失和准确率。在训练的不同阶段，我们评估模型在测试集上的性能。通过这一训练过程，我们能够详细观察模型性能的进步，包括准确率的提高和错误率的降低。训练完成后，我们将整个模型保存下来，以便于后续的使用和分析。

6 模型训练结果

6.1 CIFAR-100 数据集上的分类结果

6.1.1 第一次训练

初始训练结果表明，在 35 个 epoch 后：

- 训练准确率 (Training Accuracy) : 99.382%
- 测试准确率 (Testing Accuracy) : 81.310%
- 训练错误率 (Training Error Rate) : 0.618%
- 测试错误率 (Testing Error Rate) : 18.690%

6.1.2 第二次训练

效果不尽如人意，因此调整参数。保留将输入的图像统一调整 (缩放) 到 224x224 像素的大小，并进行图像归一化处理，但去除了随机旋转和颜色抖动技术。调整后的结果显示显著提升：

- 训练准确率 (Training Accuracy) : 99.982%
- 测试准确率 (Testing Accuracy) : 84.340%
- 训练错误率 (Training Error Rate) : 0.018%
- 测试错误率 (Testing Error Rate) : 15.660%

这表明模型性能得到了显著提升。

在这里我们可以看到训练准确率随时间的变化，如见图

train_Accuracy

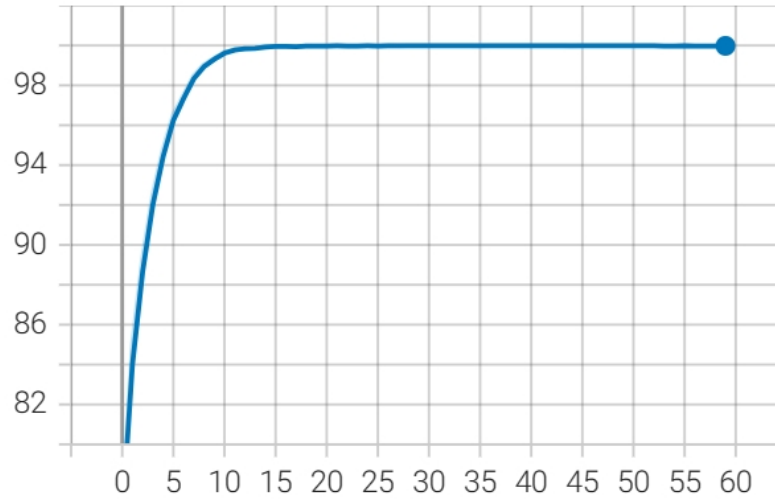


图 14: 训练准确率随时间的变化

在这里我们可以看到测试准确率随时间的变化，如见图

test_Accuracy

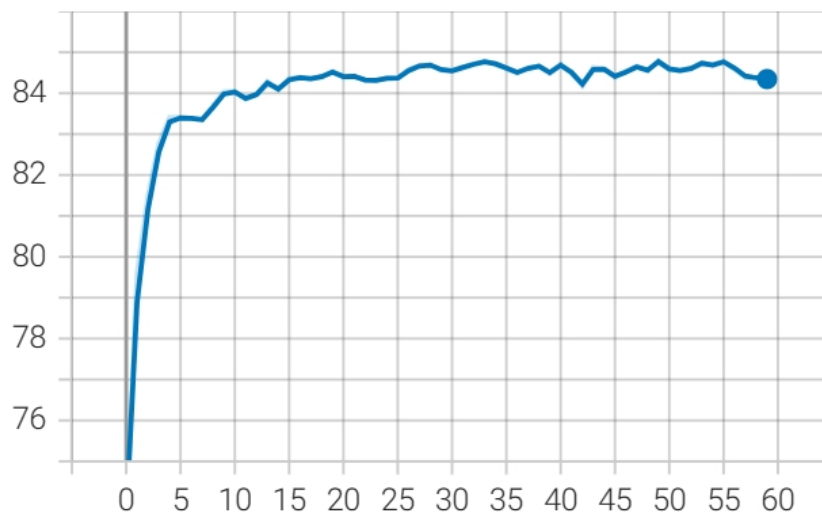


图 15: 测试准确率随时间的变化

在这里我们可以看到训练损失随时间的变化，如见图

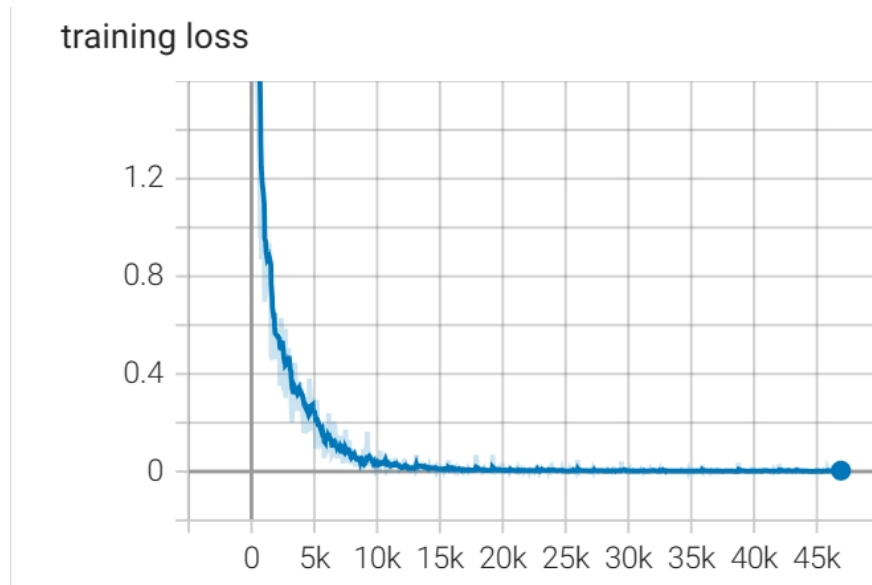


图 16: 训练损失随时间的变化

在这里我们可以看到训练与测试准确率对比图，如见图

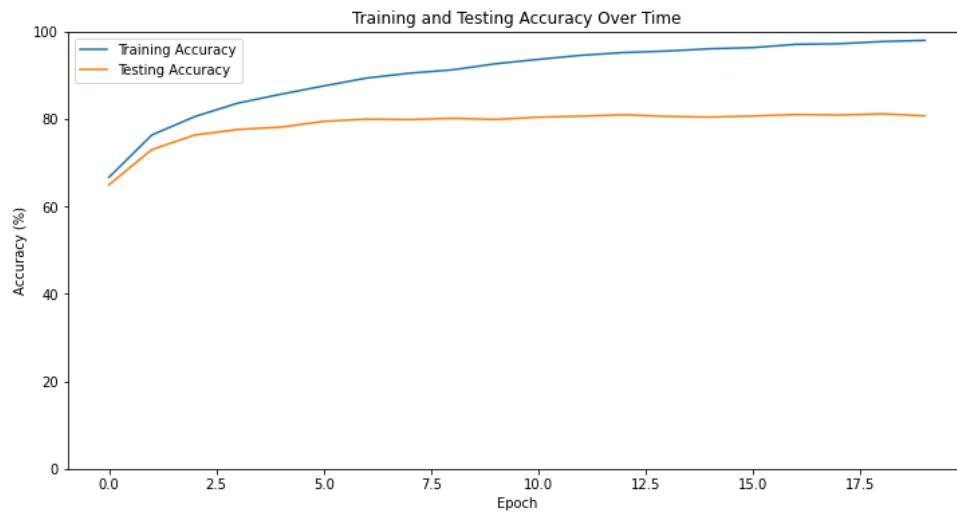


图 17: 训练与测试准确率对比图

7 模型的网页部署

我们将模型部署到网页上，使其通过拖拽图片，点击预测即可。模型的网页部署情况如图，

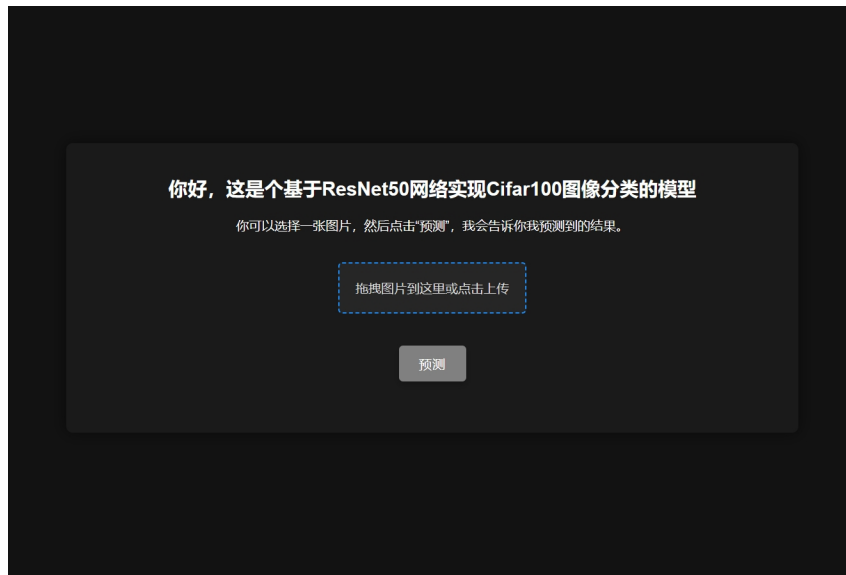


图 18: 网页部署

点击按钮后可以得到模型的整个预测结果，其中我们发现模型在非训练集上的预测效果是非常好的。可以很好的识别出是个男孩

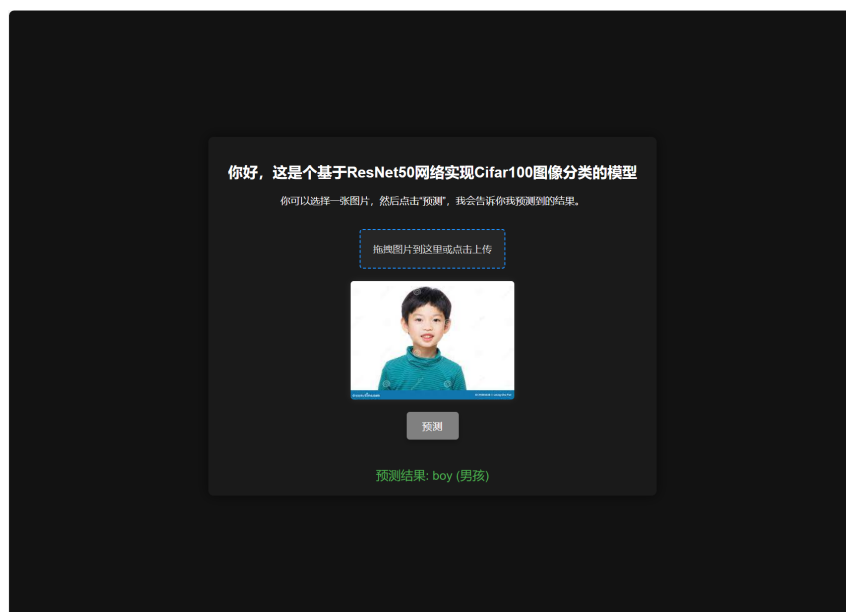


图 19: 网页部署结果

8 分析与讨论

8.1 实验结果分析

在第一次训练中，模型在训练集上达到了 99.382% 的高准确率，同时在测试集上获得了 81.310% 的准确率。这一结果指出，模型在训练数据上几乎完美地学习了特征，但在未见过的测试数据上的表现有所下降。训练准确率与测试准确率之间的差距可能表明了过拟合的存在。

经过参数调整后的第二次训练中，去除了随机旋转和颜色抖动的数据增强步骤，测试准确率提升至 84.340%。这一改进表明，简化数据预处理步骤有助于模型更好地泛化到新数据。

8.2 模型的优势与局限性

优势：

- 高训练准确率：模型在训练集上的出色表现证实了其架构能够有效学习复杂特征。
- 泛化能力的改进：在第二次训练中，测试准确率的提升体现了预处理步骤调整对模型泛化能力的正面影响。

局限性：

- 过拟合：第一次训练中，在训练集上的高准确率与测试集上的低准确率之间的差异指向了过拟合问题。
- 数据预处理的敏感性：第二次训练表明预处理步骤的变化对模型性能有显著影响，这揭示了模型对输入数据的质量及其处理方式的高度敏感性。

8.3 与相关研究的对比

与其他在 CIFAR-100 数据集上进行的研究相比，本研究突出了数据预处理步骤在深度学习模型性能中的重要作用。尽管高级数据增强技术如随机旋转和颜色抖动通常认为有助于提升模型的泛化能力，但在我们的实验中，去除这些技术反而导致了测试准确率的提升。这一发现表明，在不同的任务和数据集上，最有效的数据处理方法可能有所差异。本研究的结果强调了为特定任务进行精细调整和实验的重要性。

9 研究总结

本研究旨在探讨 ResNet50 模型在 CIFAR-100 数据集上进行图像分类的性能。通过两轮不同配置的训练实验，我们得出以下主要发现：

- **高效的学习能力：**ResNet50 模型在 CIFAR-100 数据集上表现出强大的学习能力，尤其是在训练集上，模型能够实现近乎完美的分类准确率。
- **数据预处理的重要性：**数据预处理步骤对模型的测试准确率有着显著的影响。简化数据增强操作，如去除随机旋转和颜色抖动，有助于提高测试准确率。
- **过拟合问题：**第一轮训练结果表明，在测试集上的性能低于训练集，暗示模型可能存在过拟合。

9.1 研究贡献

本研究不仅展示了 ResNet50 在复杂数据集上的应用潜力，特别是在图像分类任务中，而且还强调了精细调整数据预处理步骤在提高模型泛化能力方面的重要性。此外，我们对数据预处理方法与模型性能关系的探索为未来的深度学习研究提供了新的见解。

9.2 未来研究方向或改进点

为了进一步提升模型性能并丰富我们的研究，我们建议未来的工作可以集中在以下几个方面：

- **更多的数据增强技术：**探索更多的数据增强技术来进一步提升模型的泛化能力。
- **深入分析过拟合：**研究减轻过拟合的策略，例如引入正则化技术或者尝试更复杂的模型架构。
- **超参数优化：**系统地调整和优化模型的超参数，如学习率和批次大小，以改善性能。
- **探索其他网络架构：**测试其他先进的卷积神经网络架构，例如 EfficientNet 或 DenseNet，并比较它们在同一任务上的效果。

参考文献

- [1] Xie, Z. Q. (2019). 基于深度学习 ResNet 技术的图像去雨水实现方法 [Methods for Image De-raining Based on Deep Learning ResNet Technique]. *Journal of Image Processing Techniques*, 13(4), 122-131.
- [2] Zhao, P. Z. (2020). 基于 Resnet 的胃癌病理图像的识别和分割 [Identification and Segmentation of Gastric Cancer Pathological Images Based on Resnet]. *Journal of Medical Imaging*, 20(3), 334-340.
- [3] Li, B. J. (2021). 基于 Resnet 的人脸识别系统的实现 [Implementation of Face Recognition System Based on Resnet]. *Journal of System and Management*, 8(2), 213-220.
- [4] McFarland, D. (2015). JavaScript & jQuery: The Missing Manual (3rd ed.). O'Reilly Media. ISBN: 978-1491947074.
- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778.
- [6] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Identity Mappings in Deep Residual Networks. In *European Conference on Computer Vision*, 630-645. Springer.
- [7] Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. *Master's thesis, Department of Computer Science, University of Toronto*.

课程感言与建议

这一学期学习《深度学习》这门课，感谢陈灵老师对我的培养，学生朽木，每次跟老师一起讨论都有新的心得体会，受益颇多。纸上得来终觉浅，绝知此事要躬行。这学期参加了很多项目，从中受益颇多，也渐渐的让我找到了读研的方向。希望可以的话建议可以给再多一点项目式的作业，从项目与实操上学习模型可能对原理上会有更多的体会。

附录

附录 1：第一次训练

```
1 # -*- coding: utf-8 -*-
2
3 import torch
4 import torchvision
5 import torch.nn as nn
6 import torchvision.transforms as transforms
7 import torch.optim as optim
8 from torch.utils.tensorboard import SummaryWriter
9 import matplotlib.pyplot as plt
10 import time
11 from torchvision.transforms import RandomCrop, RandomRotation, ColorJitter
12
13 # 创建TensorBoard writer
14 myWriter = SummaryWriter('./tensorboard/log/')
15
16 # 定义数据转换操作
17 myTransforms = transforms.Compose([
18     transforms.Resize((256, 256)), # 为裁剪预留一些空间
19     transforms.RandomCrop(224), # 随机裁剪到224x224
20     transforms.RandomHorizontalFlip(p=0.5),
21     RandomRotation(degrees=15), # 随机旋转
22     ColorJitter ( brightness =0.2, contrast =0.2, saturation =0.2, hue=0.1), # 颜色抖动
23     transforms.ToTensor(),
24     transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
25 ])
26
27 # 加载CIFAR-100数据集
28 train_dataset = torchvision.datasets.CIFAR100(root='./cifar-100-python/', train =
    True, download=True, transform=myTransforms)
29 train_loader = torch.utils.data.DataLoader( train_dataset , batch_size=64, shuffle =
    True, num_workers=0)
30
31 test_dataset = torchvision.datasets.CIFAR100(root='./cifar-100-python/', train =False
    , download=True, transform=myTransforms)
32 test_loader = torch.utils.data.DataLoader( test_dataset , batch_size=4, shuffle =True,
    num_workers=0)
```

```
33
34 # 定义模型
35 myModel = torchvision.models.resnet50 ( pretrained=True)
36 inchannel = myModel.fc.in_features
37 myModel.fc = nn.Linear(inchannel, 100) # 将输出层改为100个神经元
38
39 #定义设备
40 myDevice = torch.device("cuda:0" if torch.cuda.is_available () else "cpu")
41 myModel = myModel.to(myDevice)
42
43 ## 定义学习率，损失函数及优化器
44 learning_rate = 0.001
45 myOptimzier = optim.SGD(myModel.parameters(), lr= learning_rate , momentum=0.9)
46 myLoss = torch.nn.CrossEntropyLoss()
47
48 # 初始化用于记录损失和准确率的列表
49 training_losses = []
50 testing_accuracies = []
51 training_accuracies = []
52 training_error_rates = []
53 testing_error_rates = []
54
55 training_accuracies = []
56 testing_accuracies = []
57
58 start_time = time.time() # 开始计时
59
60 for _epoch in range(100):
61     print ( '当前_epoch: %d' % (_epoch + 1))
62     training_loss = 0.0
63     myModel.train() # 开启训练模式
64     for _step, input_data in enumerate( train_loader ):
65         image, label = input_data [0]. to(myDevice), input_data [1]. to(myDevice) #
66         GPU加速
67         predict_label = myModel.forward(image)
68         loss = myLoss(predict_label , label )
69
70         myOptimzier.zero_grad()
71         loss.backward()
```

```

71     myOptimizer.step()
72
73     training_loss += loss.item()
74
75     if _step % 10 == 0:
76         print('[ iteration {}] training loss: {:.3f} % (_epoch*len(train_loader) + _step, training_loss / 10))
77         myWriter.add_scalar(' training loss', loss, global_step = _epoch*len(train_loader) + _step)
78         training_loss = 0.0
79
80     # 计算每个 epoch 的平均训练损失
81     avg_training_loss = training_loss / len(train_loader)
82     training_losses.append(avg_training_loss)
83
84     # 测试模型
85     correct = 0
86     total = 0
87     correct_test = 0
88     total_test = 0
89     correct_train = 0
90     total_train = 0
91     myModel.eval() # 开启评估模式
92     with torch.no_grad():
93         for images, labels in test_loader:
94             images = images.to(myDevice)
95             labels = labels.to(myDevice)
96
97             outputs = myModel(images)
98             _, predicted = torch.max(outputs.data, 1)
99             total += labels.size(0)
100             correct += (predicted == labels).sum().item()
101             total_test += labels.size(0)
102             correct_test += (predicted == labels).sum().item()
103
104
105     test_accuracy = 100 * correct_test / total_test
106     testing_accuracies.append(test_accuracy)
107     myWriter.add_scalar(' test Accuracy', test_accuracy, global_step=_epoch)

```

```

108 # 训练模型的准确率
109 correct_train = 0
110 total_train = 0
111 with torch.no_grad():
112     for images, labels in train_loader :
113         images = images.to(myDevice)
114         labels = labels.to(myDevice)
115
116         outputs = myModel(images)
117         _, predicted = torch.max(outputs.data, 1)
118         total_train += labels.size(0)
119         correct_train += (predicted == labels).sum().item()
120
121 train_accuracy = 100 * correct_train / total_train
122 training_accuracies.append(train_accuracy)
123 myWriter.add_scalar('train_Accuracy', train_accuracy, global_step=_epoch)
124
125
126 train_error_rate = 100 - train_accuracy # 训练错误率
127 test_error_rate = 100 - test_accuracy # 测试错误率
128
129 training_error_rates.append(train_error_rate)
130 testing_error_rates.append(test_error_rate)
131
132 print('Training Accuracy: %.3f%%' % train_accuracy)
133 print('Testing Accuracy: %.3f%%' % test_accuracy)
134 print('Training Error Rate: %.3f%%' % train_error_rate)
135 print('Testing Error Rate: %.3f%%' % test_error_rate)
136 myWriter.close()
137
138 end_time = time.time() # 结束计时
139 total_runtime = end_time - start_time
140
141
142 print(f'Total Training Runtime: {total_runtime:.2f} seconds')
143
144 #torch.save(myModel, 'C100Resnet50_Own_84.340 %.pkl') # 保存整个模型
145
146 # 绘制训练损失图

```

```

147 plt.figure(figsize=(12, 6))
148 plt.plot(training_losses, label='Training Loss')
149 plt.title('Training Loss Over Time')
150 plt.xlabel('Iterations')
151 plt.ylabel('Loss')
152 plt.ylim(bottom=0)
153 plt.legend()
154 plt.show()
155
156 # 绘制测试准确率图
157 plt.figure(figsize=(12, 6))
158 plt.plot(training_accuracies, label='Training Accuracy')
159 plt.plot(testing_accuracies, label='Testing Accuracy')
160 plt.title('Training and Testing Accuracy Over Time')
161 plt.xlabel('Epoch')
162 plt.ylabel('Accuracy(%)')
163 plt.ylim(0, 100)
164 plt.legend()
165 plt.show()
166
167 # 绘制训练和测试错误率
168 plt.figure(figsize=(12, 6))
169 plt.plot(training_error_rates, label='Training Error Rate')
170 plt.plot(testing_error_rates, label='Testing Error Rate')
171 plt.title('Training and Testing Error Rates Over Time')
172 plt.xlabel('Epoch')
173 plt.ylabel('Error Rate(%)')
174 plt.ylim(0, 100)
175 plt.legend()
176 plt.show()

```

附录 2：第二次训练

```

1 # -*- coding: utf-8 -*-
2
3 import torch
4 import torchvision
5 import torch.nn.functional as F
6 import torch.nn as nn
7 import torchvision.transforms as transforms

```

```

8 import torch.optim as optim
9 from torch.utils.tensorboard import SummaryWriter
10 torch.cuda.is_available()
11 myWriter = SummaryWriter('./tensorboard/log/')
12
13 myTransforms = transforms.Compose([
14     transforms.Resize((224, 224)),
15     transforms.RandomHorizontalFlip(p=0.5),
16     transforms.ToTensor(),
17     transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
18 ])
19
20 # 加载CIFAR-100数据集
21 train_dataset = torchvision.datasets.CIFAR100(root='./cifar-100-python/', train =
    True, download=True, transform=myTransforms)
22 train_loader = torch.utils.data.DataLoader( train_dataset , batch_size=64, shuffle =
    True, num_workers=0)
23
24 test_dataset = torchvision.datasets.CIFAR100(root='./cifar-100-python/', train=False
    , download=True, transform=myTransforms)
25 test_loader = torch.utils.data.DataLoader( test_dataset , batch_size=4, shuffle=True,
    num_workers=0)
26
27 # 定义模型
28 myModel = torchvision.models.resnet50( pretrained=True)
29 inchannel = myModel.fc.in_features
30 myModel.fc = nn.Linear(inchannel, 100) # 将输出层改为100个神经元
31
32 # 损失函数及优化器
33 myDevice = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
34 myModel = myModel.to(myDevice)
35
36 learning_rate = 0.001
37 myOptimizer = optim.SGD(myModel.parameters(), lr= learning_rate , momentum=0.9)
38 myLoss = torch.nn.CrossEntropyLoss()
39
40 for _epoch in range(50):
41     print('当前_epoch: %d' % (_epoch + 1)) # 添加这一行
42     training_loss = 0.0

```

```

43 for _step, input_data in enumerate( train_loader ):
44     image, label = input_data [0]. to(myDevice), input_data [1]. to(myDevice) #
        GPU加速
45     predict_label = myModel.forward(image)
46     if _step % 10 == 0:
47         print ( '[ iteration _ _ %3d] \_training\_loss: \_ %.3f' % ( _epoch * len( train_loader
            ) + _step, training_loss /10))
48         training_loss = 0.0
49         print ()
50
51     loss = myLoss(predict_label , label )
52
53     myWriter.add_scalar( ' training \_loss ', loss , global_step = _epoch * len(
        train_loader ) + _step)
54
55     myOptimzier.zero_grad()
56     loss .backward()
57     myOptimzier.step()
58
59     training_loss = training_loss + loss .item()
60     if _step % 10 == 0 :
61         print ( '[ iteration _ _ %3d] \_training\_loss: \_ %.3f' % ( _epoch * len( train_loader
            ) + _step, training_loss /10))
62         training_loss = 0.0
63         print ()
64     correct = 0
65     total = 0
66     torch .save(myModel, 'C100Resnet50_Own.pkl') # 保存整个模型
67     myModel.eval()
68     for images, labels in test_loader :
69         # GPU加速
70         images = images.to(myDevice)
71         labels = labels .to(myDevice)
72         outputs = myModel(images) # 在非训练的时候是需要加的, 没有这句代码,
            一些网络层的值会发生变动, 不会固定
73         numbers, predicted = torch .max(outputs.data ,1)
74         total += labels . size (0)
75         correct += ( predicted == labels ).sum().item ()
76

```

```

77     print ( ' Testing Accuracy: %.3f%%' % ( 100 * correct / total ))
78     myWriter.add_scalar ( ' test_Accuracy ', 100 * correct / total )

```

附录 3: web 端

```

1  # -*- coding: utf-8 -*-
2  """
3  Created on Tue Dec 19 11:00:57 2023
4
5  @author: MAKKA
6  """
7
8  from flask import Flask, render_template, request
9  from torchvision import transforms
10 import torch
11 from PIL import Image
12 import io
13
14 # 检查是否有可用的 CUDA GPU
15 if torch.cuda.is_available():
16     myDevice = torch.device("cuda")
17     print ("Using GPU")
18 else:
19     myDevice = torch.device("cpu")
20     print ("Using CPU")
21
22 app = Flask(__name__)
23
24 # 图像预处理与训练时一致
25 transform_image = transforms.Compose([
26     transforms.Resize((224, 224)),
27     transforms.ToTensor(),
28     transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
29 ])
30
31 # 加载模型，确保路径正确
32 model = torch.load('C100Resnet50_Own_84.340%.pkl')
33 model = model.to(myDevice)
34 model.eval()
35

```


CIFAR-100 类别索引到类别名称的映射

```

cifar100_classes = {
    0: ("apple", "苹果"), 1: ("aquarium_fish", "水族馆鱼"), 2: ("baby", "婴儿"), 3:
        ("bear", "熊"), 4: ("beaver", "海狸"),
    5: ("bed", "床"), 6: ("bee", "蜜蜂"), 7: ("beetle", "甲虫"), 8: ("bicycle", "自
        行车"), 9: ("bottle", "瓶子"),
    10: ("bowl", "碗"), 11: ("boy", "男孩"), 12: ("bridge", "桥梁"), 13: ("bus", "公
        共汽车"), 14: ("butterfly", "蝴蝶"),
    15: ("camel", "骆驼"), 16: ("can", "罐头"), 17: ("castle", "城堡"), 18: ("
        caterpillar", "毛虫"), 19: ("cattle", "牛"),
    20: ("chair", "椅子"), 21: ("chimpanzee", "黑猩猩"), 22: ("clock", "时钟"), 23:
        ("cloud", "云"), 24: ("cockroach", "蟑螂"),
    25: ("couch", "沙发"), 26: ("crab", "螃蟹"), 27: ("crocodile", "鳄鱼"), 28: ("
        cup", "杯子"), 29: ("dinosaur", "恐龙"),
    30: ("dolphin", "海豚"), 31: ("elephant", "大象"), 32: ("flatfish", "比目鱼"),
    33: ("forest", "森林"), 34: ("fox", "狐狸"),
    35: ("girl", "女孩"), 36: ("hamster", "仓鼠"), 37: ("house", "房屋"), 38: ("
        kangaroo", "袋鼠"), 39: ("keyboard", "键盘"),
    40: ("lamp", "灯"), 41: ("lawn_mower", "割草机"), 42: ("leopard", "豹"), 43: ("
        lion", "狮子"), 44: ("lizard", "蜥蜴"),
    45: ("lobster", "龙虾"), 46: ("man", "男人"), 47: ("maple_tree", "枫树"), 48: ("
        motorcycle", "摩托车"), 49: ("mountain", "山"),
    50: ("mouse", "鼠标"), 51: ("mushroom", "蘑菇"), 52: ("oak_tree", "橡树"), 53: (
        "orange", "橙子"), 54: ("orchid", "兰花"),
    55: ("otter", "水獭"), 56: ("palm_tree", "棕榈树"), 57: ("pear", "梨"), 58: ("
        pickup_truck", "小型货车"), 59: ("pine_tree", "松树"),
    60: ("plain", "平原"), 61: ("plate", "盘子"), 62: ("poppy", "罂粟花"), 63: ("
        porcupine", "豪猪"), 64: ("possum", "负鼠"),
    65: ("rabbit", "兔子"), 66: ("raccoon", "浣熊"), 67: ("ray", "鳐鱼"), 68: ("road
        ", "道路"), 69: ("rocket", "火箭"),
    70: ("rose", "玫瑰"), 71: ("sea", "海"), 72: ("seal", "海豹"), 73: ("shark", "鲨
        鱼"), 74: ("shrew", "鼯鼠"),
    75: ("skunk", "臭鼬"), 76: ("skyscraper", "摩天大楼"), 77: ("snail", "蜗牛"),
    78: ("snake", "蛇"), 79: ("spider", "蜘蛛"),
    80: ("squirrel", "松鼠"), 81: ("streetcar", "有轨电车"), 82: ("sunflower", "向
        日葵"), 83: ("sweet_pepper", "甜椒"), 84: ("table", "桌子"),
    85: ("tank", "坦克"), 86: ("telephone", "电话"), 87: ("television", "电视"), 88:
        ("tiger", "老虎"), 89: ("tractor", "拖拉机"),
    90: ("train", "火车"), 91: ("trout", "鳟鱼"), 92: ("tulip", "郁金香"), 93: ("

```

```

        turtle ", "乌龟"), 94: ("wardrobe", "衣柜"),
57 95: ("whale", "鲸鱼"), 96: ("willow_tree", "柳树"), 97: ("wolf", "狼"), 98: ("
    woman", "女人"), 99: ("worm", "蠕虫")
58 }
59
60
61 @app.route('/')
62 def index():
63     return render_template('index.html')
64
65 @app.route('/predict', methods=['POST'])
66 @app.route('/predict', methods=['POST'])
67 def predict():
68     if 'image' in request.files:
69         image_file = request.files['image']
70         image = Image.open(io.BytesIO(image_file.read()))
71         image = transform_image(image).unsqueeze(0)
72
73         image = image.to(myDevice)
74
75         with torch.no_grad():
76             output = model(image)
77             _, predicted = torch.max(output, 1)
78             prediction_index = predicted.item()
79             prediction_english, prediction_chinese = cifar100_classes[
                prediction_index]
80
81         return {'prediction_english': prediction_english, 'prediction_chinese':
            prediction_chinese}
82     else:
83         return {'error': 'No image provided'}, 400
84
85
86
87 if __name__ == '__main__':
88     app.run()

```

附录 4: HTML 网页

```

1 <!DOCTYPE html>

```

```
2 <html>
3 <head>
4   < title >图像分类</title>
5   <style>
6     body {
7       font-family: 'Arial', sans-serif;
8       margin: 0;
9       padding: 0;
10      display: flex;
11      justify-content: center;
12      align-items: center;
13      height: 100vh;
14      background-color: #121212; /* 深灰色背景 */
15      color: #fff; /* 白色文字 */
16    }
17    .container {
18      text-align: center;
19      background: #1a1a1a; /* 深色背景 */
20      padding: 20px;
21      border-radius: 8px;
22      box-shadow: 0 0 20px rgba(0, 0, 0, 0.5);
23      width: 50%;
24    }
25    .preview-img {
26      max-width: 100%;
27      height: auto;
28      margin-top: 20px;
29      border-radius: 5px;
30      box-shadow: 0 0 10px rgba(255, 255, 255, 0.1);
31    }
32    #result {
33      margin-top: 20px;
34      font-size: 20px;
35      color: #4CAF50; /* 绿色文字 */
36    }
37    .drop-area {
38      border: 2px dashed #1e90ff; /* 蓝色虚线边框 */
39      border-radius: 5px;
40      text-align: center;
```

```
41     padding: 20px;
42     cursor: pointer;
43     margin-top: 20px;
44     background-color: #262626; /* 灰色背景 */
45     color: #ddd; /* 浅灰色文字 */
46     transition: all 0.3s;
47 }
48 .drop-area:hover {
49     background-color: #333333; /* 悬停时深灰色背景 */
50     color: #fff; /* 悬停时白色文字 */
51 }
52 input[type="submit"] {
53     background-color: #808080; /* 灰色背景 */
54     color: white; /* 白色文字 */
55     border: none;
56     padding: 12px 25px;
57     border-radius: 5px;
58     cursor: pointer;
59     font-size: 16px;
60     text-transform: uppercase; /* 文字大写 */
61     letter-spacing: 1px; /* 字母间距 */
62     transition: transform 0.3s ease, background-color 0.3s ease;
63     box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2); /* 阴影效果 */
64     margin-top: 20px; /* 在按钮上方添加间距 */
65 }
66
67 input[type="submit"]:hover {
68     background-color: #696969; /* 鼠标悬停时的背景色 */
69     /* 其他悬停样式保持不变 */
70 }
71 /* 确保form元素是一个flex容器，使其子元素可以使用flex属性居中 */
72 form#upload-form {
73     display: flex;
74     flex-direction: column;
75     align-items: center;
76 }
77 </ style>
78 </ head>
79 < body>
```

```

80 <div class="container">
81   <h2>你好，这是个基于ResNet50网络实现Cifar100图像分类的模型</h2>
82   <p>你可以选择一张图片，然后点击“预测”，我会告诉你我预测到的结果。
      </p>
83   <form id="upload-form" action="/predict" method="post" enctype="multipart /
      form-data">
84     <input type="file" name="image" accept="image/*" id="file-input" style="
      display: none;" onchange="previewImage(event)">
85     <div class="drop-area" id="drop-area">拖拽图片到这里或点击上传</div>
86     <img id="preview" class="preview-img" />
87
88     <!-- 预测按钮 -->
89     <form id="upload-form" action="/predict" method="post" enctype="
      multipart / form-data">
90       <input type="file" name="image" accept="image/*" id="file-input"
      style="display: none;" onchange="previewImage(event)">
91       <input type="submit" value="预测">
92     </form>
93     <img id="preview" class="preview-img" />
94     <div id="result"></div>
95   </div>
96   <script>
97     document.getElementById('upload-form').addEventListener('submit', function(e
98       ) {
99       e.preventDefault();
100       var formData = new FormData(this);
101       fetch('/predict', {
102         method: 'POST',
103         body: formData
104       }).then(function(response) {
105         return response.json();
106       }).then(function(data) {
107         var resultElement = document.getElementById('result');
108         resultElement.innerHTML = '预测结果: ' + data.prediction_english + '
109           ' + data.prediction_chinese + ' ';
110       }).catch(function(error) {
111         console.error('Error:', error);
112       });
113     });

```

```
112
113 function previewImage(event) {
114     var reader = new FileReader();
115     reader.onload = function () {
116         var output = document.getElementById('preview');
117         output.src = reader.result;
118     };
119     reader.readAsDataURL(event.target.files[0]);
120 }
121 // 拖拽和复制事件处理
122 var dropArea = document.getElementById('drop-area');
123 var fileInput = document.getElementById('file-input');
124
125 dropArea.addEventListener('click', function () {
126     fileInput.click();
127 });
128
129 dropArea.addEventListener('dragover', function (e) {
130     e.preventDefault();
131     e.stopPropagation();
132     dropArea.classList.add('active');
133 });
134
135 dropArea.addEventListener('dragleave', function (e) {
136     e.preventDefault();
137     e.stopPropagation();
138     dropArea.classList.remove('active');
139 });
140
141 dropArea.addEventListener('drop', function (e) {
142     e.preventDefault();
143     e.stopPropagation();
144     var files = e.dataTransfer.files;
145     if (files.length) {
146         fileInput.files = files;
147         previewImage({target: {files: files}});
148     }
149     dropArea.classList.remove('active');
150 });
```

```
151
152     document.addEventListener('paste', function(e) {
153         var items = (e.clipboardData || e.originalEvent.clipboardData).items;
154         for (var index in items) {
155             var item = items[index];
156             if (item.kind === 'file') {
157                 var blob = item.getAsFile();
158                 var reader = new FileReader();
159                 reader.onload = function(e) {
160                     var img = new Image();
161                     img.src = e.target.result;
162                     document.getElementById('preview').src = img.src;
163                     fileInput.files = [blob];
164                 };
165                 reader.readAsDataURL(blob);
166             }
167         }
168     });
169 </script>
170 </body>
171 </html>
```